JAVA EDITION · 30 DAYS · 2-5 HRS/DAY

Data Structures to System Design

A day-by-day plan to crack Google, Microsoft and Amazon interviews, with a live Java demo every day and a resume-grade distributed systems project.

30	161	30	20
days	LeetCode problems	live Java demos	coding patterns

Prepared for **Shubham** · Day 1: Mon 02 Nov 2026 · Day 30: Tue 01 Dec

Version 2 · updated 01 Nov 2026 (added matrices, cyclic sort, math & number theory, randomised algorithms, core Java/JVM, API design & security, distributed transactions, 20-pattern table, Java cheat sheet, interview loops)

FLAGSHIP PROJECT

NexusKV: A distributed, fault-tolerant key-value store built from scratch in Java

It is built out of the exact data structures interviewers ask about (skip list, hash map, LRU, heap, Bloom filter, consistent hashing ring) and the exact system-design topics they ask about (LSM storage, WAL, sharding, replication, leader election). Every interview question about your resume becomes a chance to show depth.

How this plan works

- Every day has: **concepts** to learn, a **live Java demo** you type and run, **LeetCode problems** (easy to hard, linked), a **NexusKV project task** on build days, and **sticky notes** to revise.
- Each topic is taught one at a time in chat: tell me *'Start Day N'* and I teach it in depth with full Java code, dry runs and interview tips, then update the sticky-notes board.
- On short (2-hour) days, do the concepts, demo and the first 3 problems. Catch up the rest on weekends.
- Rule of 25: if stuck on a problem for 25 minutes, read the hint/solution, then re-solve it from scratch 2 days later.
- Everything is in Java 21 (records, var, switch expressions, streams where it helps; plain loops in interview code).

Daily time split

2-hour day

- 40 min: concept lesson + notes
- 20 min: live Java demo (type it yourself)
- 50 min: 2-3 core problems
- 10 min: sticky-note revision

5-hour day

- 60 min: concept lesson + notes
- 30 min: live Java demo + variations
- 2 h: all listed problems (timer: 25 min each)
- 60 min: NexusKV project task
- 30 min: revise sticky notes + re-solve 1 old problem

The 30 days at a glance

Day	Date	Topic	Build
1	Mon 02 Nov	Java for DSA + Complexity Analysis	■
2	Tue 03 Nov	Arrays & Strings: Two Pointers, Sliding Window, Prefix Sums	
3	Wed 04 Nov	Hashing & HashMap Internals	
4	Thu 05 Nov	Recursion & Backtracking	
5	Fri 06 Nov	Sorting & Binary Search	
6	Sat 07 Nov	Linked Lists	■
7	Sun 08 Nov	Stacks, Queues, Deques + Week 1 Review	■
8	Mon 09 Nov	Binary Trees	
9	Tue 10 Nov	BST & Balanced Trees (AVL, Red-Black, B-Tree)	
10	Wed 11 Nov	Heaps & Priority Queues	■
11	Thu 12 Nov	Tries & String Algorithms	■
12	Fri 13 Nov	Graphs I: Representation, BFS, DFS, Topological Sort	
13	Sat 14 Nov	Graphs II: Shortest Paths, MST, Union-Find	
14	Sun 15 Nov	Week 2 Review + NexusKV Storage Engine	■
15	Mon 16 Nov	Greedy & Intervals	
16	Tue 17 Nov	Dynamic Programming I: 1D & Knapsack	
17	Wed 18 Nov	Dynamic Programming II: 2D, Strings, Grids	
18	Thu 19 Nov	DP III (Interval, Tree, Bitmask) + Bit Manipulation	■
19	Fri 20 Nov	Range Query Structures: Segment Tree, Fenwick Tree, Sparse Table	
20	Sat 21 Nov	Advanced Graphs + Union of Patterns	

Day	Date	Topic	Build
21	Sun 22 Nov	Week 3 Review + Mock Contest	■
22	Mon 23 Nov	OOP, SOLID & Design Patterns (Low-Level Design I)	■
23	Tue 24 Nov	Java Concurrency + LLD Case Studies	■
24	Wed 25 Nov	System Design Fundamentals (HLD I)	
25	Thu 26 Nov	Databases, Storage & Data Distribution (HLD II)	■
26	Fri 27 Nov	Distributed Systems & Microservices (HLD III)	■
27	Sat 28 Nov	HLD Case Studies	
28	Sun 29 Nov	NexusKV: Production Polish	■
29	Mon 30 Nov	Resume, Behavioural & Company-Specific Prep	■
30	Tue 01 Dec	Full Mock Interview Day + Final Revision	

WEEK 1

Foundations & Linear Structures

Java for DSA, complexity, arrays, hashing, recursion, sorting, linked lists, stacks/queues

DAY 1 · MON 02 NOV

Java for DSA + Complexity Analysis

Concepts

- Big-O, Big-Omega, Big-Theta; best/avg/worst case; amortized analysis
- Time vs space; counting operations; recursion-tree & Master theorem
- JVM memory: stack vs heap, references, primitives vs wrappers, autoboxing cost
- Collections Framework map: List / Set / Map / Queue / Deque and when to use each
- Fast I/O (BufferedReader, StringBuilder), Comparable vs Comparator, generics
- Core Java interview set: String/StringBuilder/StringBuffer, immutability, String pool, final/finally/finalize
- JVM & GC: heap generations, G1/ZGC basics, OutOfMemoryError vs StackOverflowError
- Java 8-21 features: lambdas, streams, Optional, records, sealed classes, var; checked vs unchecked exceptions

LIVE JAVA DEMO

Benchmark ArrayList vs LinkedList vs ArrayDeque insert/get with System.nanoTime(); watch O(1) vs O(n) appear in real numbers.

Problems

- Two Sum — Easy
- Contains Duplicate — Easy
- Fizz Buzz — Easy
- Richest Customer Wealth — Easy

NexusKV build: Create the GitHub repo + Maven multi-module skeleton for NexusKV; set up JUnit 5.

STICKY NOTES

- Drop constants and lower terms: O(2n + 5) = O(n)
- Amortized O(1): ArrayList doubles capacity, so n adds cost about 3n work
- Integer caching only covers -128..127: compare wrappers with equals(), never ==
- Interviewers expect you to state time AND space before coding
- StackOverflowError = recursion too deep; raise -Xss or go iterative

DAY 2 · TUE 03 NOV

Arrays & Strings: Two Pointers, Sliding Window, Prefix Sums

Concepts

- Static vs dynamic arrays; 2D arrays; String immutability & String pool
- Two pointers (opposite ends, same direction, fast/slow)
- Sliding window: fixed size and variable size with a HashMap of counts
- Prefix sum & difference array; Kadane's algorithm
- StringBuilder, char[] frequency tables (int[26])
- Matrix traversal: spiral, rotate 90 degrees in place (transpose + reverse), set-zeroes with O(1) space
- Cyclic sort pattern for numbers in range 1..n (missing / duplicate numbers)

LIVE JAVA DEMO

Animate a sliding window in the console: print window bounds and running sum at every step for 'max sum subarray of size k'.

Problems

- Best Time to Buy and Sell Stock — Easy
- Container With Most Water — Medium

- 3Sum — Medium
- Longest Substring Without Repeating Characters — Medium
- Maximum Subarray — Medium
- Product of Array Except Self — Medium
- Minimum Window Substring — Hard
- Trapping Rain Water — Hard
- Spiral Matrix — Medium
- Rotate Image — Medium
- Set Matrix Zeroes — Medium
- Missing Number — Easy
- Find All Duplicates in an Array — Medium
- First Missing Positive — Hard

STICKY NOTES

• Sorted array + pair/triplet target = two pointers
• 'Longest/shortest substring with condition' = variable sliding window
• Subarray sum = k: prefix sum + HashMap
• Kadane: cur = max(x, cur + x); best = max(best, cur)
• Numbers in 1..n => cyclic sort: put x at index x-1, then scan

DAY 3 · WED 04 NOV

Hashing & HashMap Internals

Concepts

- Hash functions, load factor, rehashing, collision handling (chaining vs open addressing)
- Java HashMap internals: table of buckets, treeification to red-black tree at 8 nodes
- equals()/hashCode() contract; why mutable keys break maps
- HashSet, LinkedHashMap (insertion/access order), TreeMap (sorted, O(log n))
- Frequency counting, grouping by canonical key, rolling hash intro

LIVE JAVA DEMO

Implement MyHashMap from scratch with chaining and resize; then break it with a bad hashCode() and measure the slowdown.

Problems

- Valid Anagram — Easy
- Group Anagrams — Medium
- Top K Frequent Elements — Medium
- Longest Consecutive Sequence — Medium
- Subarray Sum Equals K — Medium
- Design HashMap — Easy

STICKY NOTES

• Average O(1), worst O(n) (O(log n) after Java 8 treeify)
• Override equals() => must override hashCode()
• Default load factor 0.75; capacity always a power of 2
• Group by key: sorted string or int[26] signature

DAY 4 · THU 05 NOV

Recursion & Backtracking

Concepts

- Call stack, base case, recursion tree, tail recursion (Java has no TCO)
- Subsets, permutations, combinations templates
- Backtracking: choose -> explore -> un-choose; pruning
- N-Queens, Sudoku, word search on grids
- Complexity of exponential searches (2^n, n!)

LIVE JAVA DEMO

Print the recursion tree with indentation for permutations of 'ABC', then add pruning and count saved calls.

Problems

Subsets	Medium
Permutations	Medium
Combination Sum	Medium
Generate Parentheses	Medium
Word Search	Medium
N-Queens	Hard

STICKY NOTES

• One template solves subsets/permutations/combinations: only the loop start and 'used' check change
• Always undo the choice after the recursive call
• Copy the path when you add it: new ArrayList<>(path)
• Prune early to cut whole subtrees

DAY 5 · FRI 06 NOV

Sorting & Binary Search

Concepts

- Bubble/Selection/Insertion (why O(n^2)); Merge sort (stable, O(n log n)); Quick sort (pivot, partition)
- Counting sort, radix sort, bucket sort; stability
- Arrays.sort: dual-pivot quicksort for primitives, TimSort for objects
- Binary search templates: exact, lower bound, upper bound
- Binary search on the answer (min capacity, Koko bananas), rotated arrays
- Divide and conquer: count inversions with merge sort, fast power x^n in O(log n)

LIVE JAVA DEMO

Visual merge sort that prints each split and merge level; then race it against Arrays.sort on 10 million ints.

Problems

Binary Search	Easy
Search in Rotated Sorted Array	Medium
Find Minimum in Rotated Sorted Array	Medium
Koko Eating Bananas	Medium
Sort Colors	Medium
Kth Largest Element in an Array	Medium
Median of Two Sorted Arrays	Hard
Pow(x, n)	Medium
Search a 2D Matrix	Medium

STICKY NOTES

• mid = lo + (hi - lo) / 2 avoids overflow
• Monotonic yes/no condition over a range => binary search on answer
• Quickselect finds kth element in average O(n)
• Merge sort is the go-to for linked lists and counting inversions

DAY 6 · SAT 07 NOV

Linked Lists

Concepts

- Singly, doubly, circular lists; sentinel/dummy nodes
- Reverse (iterative + recursive), merge, find middle
- Fast & slow pointers: cycle detection (Floyd), cycle start
- Deep copy with random pointers
- LRU cache = HashMap + doubly linked list

LIVE JAVA DEMO

Build a DoublyLinkedList with toString() that draws arrows (1 <-> 2 <-> 3), then build an LRU cache on top and show evictions live.

Problems

Reverse Linked List	Easy
Merge Two Sorted Lists	Easy
Linked List Cycle	Easy
Remove Nth Node From End of List	Medium
Reorder List	Medium
Copy List with Random Pointer	Medium
LRU Cache	Medium
Reverse Nodes in k-Group	Hard

NexusKV build: NexusKV: implement the LRU read cache (HashMap + DLL) with unit tests.

STICKY NOTES

• Dummy head removes edge cases for deleting the first node
• Fast/slow: fast moves 2, slow 1; they meet inside a cycle
• Reverse: prev=null; while(cur){next=cur.next; cur.next=prev; prev=cur; cur=next}
• LRU get/put are both O(1)

DAY 7 · SUN 08 NOV

Stacks, Queues, Deques + Week 1 Review

Concepts

- Stack (use ArrayDeque, not Stack), queue, circular queue, deque
- Monotonic stack: next greater element, histogram
- Monotonic deque: sliding window maximum
- Expression evaluation, valid parentheses, min stack
- Week 1 review: 10 mixed problems timed at 25 min each
- Implement queue using stacks and stack using queues (asked as warm-ups)

LIVE JAVA DEMO

Monotonic stack trace for 'Daily Temperatures': print the stack after every push and pop.

Problems

Valid Parentheses	Easy

- Min Stack — **Medium**
- Daily Temperatures — **Medium**
- Evaluate Reverse Polish Notation — **Medium**
- Sliding Window Maximum — **Hard**
- Largest Rectangle in Histogram — **Hard**
- Implement Queue using Stacks — **Easy**
- Design Circular Queue — **Medium**

NexusKV build: NexusKV: in-memory store with GET/PUT/DELETE and a skip-list memtable (ordered keys).

STICKY NOTES

• Deque st = new ArrayDeque<>(); is the modern stack
• 'Next greater/smaller' = monotonic stack, O(n)
• Sliding window max = deque of indices, decreasing values
• Skip list: O(log n) expected search with simple code (used in Redis & LevelDB)

WEEK 2

Trees, Heaps, Tries & Graphs

Hierarchical and network structures plus the classic graph algorithms

DAY 8 · MON 09 NOV

Binary Trees

Concepts

- Terminology: height, depth, full/complete/perfect/balanced
- DFS traversals: pre/in/post order (recursive + iterative)
- BFS level-order with a queue; zigzag; right-side view
- Return-value recursion: height, diameter, max path sum
- Serialize/deserialize, LCA, construct from traversals

LIVE JAVA DEMO

Pretty-print a binary tree sideways in the console and show all 4 traversal orders for the same tree.

Problems

Maximum Depth of Binary Tree	Easy
Invert Binary Tree	Easy
Diameter of Binary Tree	Easy
Binary Tree Level Order Traversal	Medium
Binary Tree Right Side View	Medium
Lowest Common Ancestor of a Binary Tree	Medium
Binary Tree Maximum Path Sum	Hard
Serialize and Deserialize Binary Tree	Hard

STICKY NOTES

• Most tree problems: solve for left, solve for right, combine
• Level-order: record queue.size() before the inner loop
• Global answer (diameter, path sum) + returned value (height) is a common pair
• Iterative inorder uses an explicit stack

DAY 9 · TUE 10 NOV

BST & Balanced Trees (AVL, Red-Black, B-Tree)

Concepts

- BST insert/search/delete (3 deletion cases)
- Validate BST with min/max bounds; inorder = sorted
- AVL rotations (LL, RR, LR, RL); Red-Black properties (concept level)
- TreeMap/TreeSet API: floorKey, ceilingKey, headMap, subMap
- B-Tree / B+Tree: why databases index with them

LIVE JAVA DEMO

Insert 1..7 into a plain BST (degenerates to a list) vs an AVL tree; print heights to show O(n) vs O(log n).

Problems

Validate Binary Search Tree	Medium
Kth Smallest Element in a BST	Medium
Lowest Common Ancestor of a Binary Search Tree	Medium
Delete Node in a BST	Medium
Convert Sorted Array to Binary Search Tree	Easy

- My Calendar I — Medium

STICKY NOTES

• TreeMap is a red-black tree: all ops O(log n)
• floorKey/ceilingKey solve 'closest value' problems instantly
• Validate BST with (min,max) bounds, not just parent comparison
• B+Trees keep data in leaves linked for fast range scans

DAY 10 · WED 11 NOV

Heaps & Priority Queues

Concepts

- Binary heap as an array: parent (i-1)/2, children 2i+1, 2i+2
- Sift up/down; heapify in O(n); heap sort
- PriorityQueue with comparators; min-heap vs max-heap
- Top-K pattern, K-way merge, two heaps for running median
- Scheduling problems (task scheduler, meeting rooms)

LIVE JAVA DEMO

Implement MinHeap from scratch and print the array after every insert/poll, side by side with its tree shape.

Problems

- Kth Largest Element in a Stream — Easy
- Last Stone Weight — Easy
- K Closest Points to Origin — Medium
- Task Scheduler — Medium
- Merge k Sorted Lists — Hard
- Find Median from Data Stream — Hard

NexusKV build: NexusKV: K-way merge iterator (heap) to merge sorted runs.

STICKY NOTES

• Top K largest => min-heap of size K (O(n log K))
• Max-heap: new PriorityQueue<>(Collections.reverseOrder())
• Running median: max-heap low half + min-heap high half
• Building a heap from an array is O(n), not O(n log n)

DAY 11 · THU 12 NOV

Tries & String Algorithms

Concepts

- Trie node (children[26] or HashMap), insert/search/startsWith
- Autocomplete, word dictionary with wildcards, word search II
- Pattern matching: KMP (LPS array), Rabin-Karp (rolling hash), Z-algorithm
- Palindromes: expand-around-center, Manacher (overview)
- Bloom filter: probabilistic membership with k hash functions

LIVE JAVA DEMO

Build an autocomplete engine over 10,000 words with a Trie; type a prefix and get top suggestions in microseconds.

Problems

- Implement Trie (Prefix Tree) — Medium
- Design Add and Search Words Data Structure — Medium

- Longest Palindromic Substring — Medium
- Find the Index of the First Occurrence in a String — Easy
- Word Search II — Hard

NexusKV build: NexusKV: Bloom filter per data file to skip disk reads for missing keys.

STICKY NOTES

• Trie search is O(length of word), independent of dictionary size
• KMP never moves the text pointer backwards: O(n + m)
• Bloom filter: false positives possible, false negatives never
• Rabin-Karp: compare hashes first, characters only on a hash match

DAY 12 · FRI 13 NOV

Graphs I: Representation, BFS, DFS, Topological Sort

Concepts

- Adjacency list vs matrix vs edge list; directed/undirected/weighted
- BFS (shortest path in unweighted graphs), DFS (connectivity, cycles)
- Grid as a graph: islands, flood fill, multi-source BFS
- Cycle detection: undirected (parent) and directed (colours)
- Topological sort: Kahn's (BFS, in-degree) and DFS post-order
- Multi-source BFS on grids and implicit graphs (word ladder, open the lock)

LIVE JAVA DEMO

Grid BFS that prints the board after every wave for 'Rotting Oranges', so you watch the multi-source BFS spread.

Problems

- Number of Islands — Medium
- Clone Graph — Medium
- Rotting Oranges — Medium
- Pacific Atlantic Water Flow — Medium
- Course Schedule — Medium
- Course Schedule II — Medium
- Word Ladder — Hard
- Open the Lock — Medium
- Surrounded Regions — Medium

STICKY NOTES

• Unweighted shortest path = BFS
• Dependencies/ordering = topological sort; if fewer than n nodes come out, there is a cycle
• Mark visited when you ENQUEUE, not when you dequeue
• Grid neighbours: int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}}

DAY 13 · SAT 14 NOV

Graphs II: Shortest Paths, MST, Union-Find

Concepts

- Dijkstra with PriorityQueue (non-negative weights)
- Bellman-Ford (negative edges, k-stops), Floyd-Warshall (all pairs)
- 0-1 BFS with deque
- Union-Find with path compression + union by rank
- Minimum spanning tree: Kruskal (sort + DSU), Prim (heap)

LIVE JAVA DEMO

Dijkstra on a small city map of Chennai areas; print the PQ and the distance table after every relaxation.

Problems

Network Delay Time	Medium
Cheapest Flights Within K Stops	Medium
Number of Provinces	Medium
Redundant Connection	Medium
Min Cost to Connect All Points	Medium
Path With Minimum Effort	Medium
Swim in Rising Water	Hard

STICKY NOTES

• Dijkstra: skip stale PQ entries (if d > dist[u]) continue;
• Negative weights => Bellman-Ford, never Dijkstra
• DSU with both optimisations is almost O(1) per op (inverse Ackermann)
• Kruskal = sort edges + DSU; Prim = PQ from a start node

DAY 14 · SUN 15 NOV

Week 2 Review + NexusKV Storage Engine

Concepts

- Timed mock: 2 medium + 1 hard in 90 minutes
- Revise all sticky notes from Days 8-13
- LSM-tree design: memtable -> immutable memtable -> SSTable files
- Write-ahead log (WAL) for durability and crash recovery
- Java NIO: FileChannel, ByteBuffer, fsync (force)

LIVE JAVA DEMO

Kill the NexusKV process mid-write, restart it, and watch the WAL replay recover every key.

Problems

Time Based Key-Value Store	Medium
Design Twitter	Medium
Snapshot Array	Medium

NexusKV build: NexusKV: WAL + flush memtable to sorted SSTable files with an index block and Bloom filter.

STICKY NOTES

• LSM trees turn random writes into sequential writes
• WAL first, memtable second: that order is what makes writes durable
• SSTables are immutable; deletes are tombstones
• Read path: cache -> memtable -> SSTables newest to oldest (Bloom filter first)

WEEK 3

Advanced Algorithms

Greedy, the four flavours of DP, bit tricks, segment/Fenwick trees, advanced graphs

DAY 15 · MON 16 NOV

Greedy & Intervals

Concepts

- Greedy choice property and exchange argument (how to prove greedy is correct)
- Interval patterns: merge, insert, non-overlapping, meeting rooms
- Jump game, gas station, partition labels
- Sweep line with sorted events
- When greedy fails (coin change with arbitrary coins)

LIVE JAVA DEMO

Sweep line over meeting intervals: print active meetings at every event to find minimum rooms.

Problems

Merge Intervals	Medium
Insert Interval	Medium
Non-overlapping Intervals	Medium
Jump Game	Medium
Gas Station	Medium
Partition Labels	Medium
Minimum Number of Arrows to Burst Balloons	Medium

STICKY NOTES

• Intervals: sort by start (merging) or by end (max non-overlapping)
• Meeting rooms = min-heap of end times, or sweep line
• Greedy needs a proof; if you can find a counter-example, use DP
• Sort with Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]))

DAY 16 · TUE 17 NOV

Dynamic Programming I: 1D & Knapsack

Concepts

- Overlapping subproblems + optimal substructure
- Top-down memoization vs bottom-up tabulation vs space optimisation
- The 5 steps: state, transition, base case, order, answer
- 1D DP: climbing stairs, house robber, decode ways
- 0/1 knapsack, unbounded knapsack, subset sum, coin change

LIVE JAVA DEMO

Fibonacci three ways (naive, memo, table) with call counters: 1.6 billion calls vs 90 for n=45.

Problems

Climbing Stairs	Easy
House Robber	Medium
House Robber II	Medium
Decode Ways	Medium
Coin Change	Medium
Partition Equal Subset Sum	Medium

- Longest Increasing Subsequence — Medium
- Word Break — Medium

STICKY NOTES

• Write the recursion first, then add memo, then convert to a table
• 0/1 knapsack 1D: loop capacity BACKWARDS
• Unbounded knapsack 1D: loop capacity FORWARDS
• LIS in O(n log n): patience sorting with binary search

DAY 17 · WED 18 NOV

Dynamic Programming II: 2D, Strings, Grids

Concepts

- Grid paths with obstacles; minimum path sum
- LCS, edit distance, distinct subsequences
- Palindromic substrings/subsequences
- Stock problems as state machines
- Rolling arrays to reduce 2D space to 1D

LIVE JAVA DEMO

Print the full edit-distance table for 'horse' -> 'ros' and trace back the actual edit operations.

Problems

- Unique Paths — Medium
- Minimum Path Sum — Medium
- Longest Common Subsequence — Medium
- Edit Distance — Medium
- Palindromic Substrings — Medium
- Best Time to Buy and Sell Stock with Cooldown — Medium
- Distinct Subsequences — Hard
- Regular Expression Matching — Hard

STICKY NOTES

• Two strings => dp[i][j] over prefixes of length i and j
• Edit distance: 1 + min(insert, delete, replace) when chars differ
• Stock problems: one state per (day, holding?, transactions left)
• Draw the table by hand for a tiny input before coding

DAY 18 · THU 19 NOV

DP III (Interval, Tree, Bitmask) + Bit Manipulation

Concepts

- Interval DP: burst balloons, matrix chain
- DP on trees: house robber III, max path
- Bitmask DP: travelling salesman, assign tasks
- Bit tricks: x & (x-1), x & -x, XOR properties, masks
- Counting bits, single number, subsets via bitmask
- Math & number theory: GCD/LCM (Euclid), Sieve of Eratosthenes, modular arithmetic (1e9+7), modular inverse, nCr
- Randomised algorithms (Google favourite): reservoir sampling, weighted random pick, Fisher-Yates shuffle

LIVE JAVA DEMO

Enumerate all subsets of {a,b,c,d} with bitmasks and print each mask in binary next to its subset.

Problems

Single Number	Easy
Counting Bits	Easy
Number of 1 Bits	Easy
House Robber III	Medium
Target Sum	Medium
Burst Balloons	Hard
Shortest Path Visiting All Nodes	Hard
Count Primes	Medium
Linked List Random Node	Medium
Random Pick with Weight	Medium
Insert Delete GetRandom O(1)	Medium

NexusKV build: NexusKV: background compaction (merge SSTables with the heap iterator, drop tombstones).

STICKY NOTES

• a ^ a = 0 and a ^ 0 = a: XOR finds the single element
• x & (x - 1) clears the lowest set bit
• Interval DP: loop by LENGTH of interval, outermost
• n <= 20 with 'visit all' => bitmask DP
• Take % MOD after every multiply; use long for the intermediate

DAY 19 · FRI 20 NOV

Range Query Structures: Segment Tree, Fenwick Tree, Sparse Table

Concepts

- Segment tree: build, point update, range query, O(log n)
- Lazy propagation for range updates
- Fenwick tree (Binary Indexed Tree): prefix sums with i & -i
- Sparse table for O(1) range min on static arrays
- Ordered statistics; coordinate compression

LIVE JAVA DEMO

Segment tree over [2,1,5,3,4]: print the tree array after build and after each update, and trace a range-sum query.

Problems

Range Sum Query - Mutable	Medium
Count of Smaller Numbers After Self	Hard
Range Sum Query 2D - Immutable	Medium
My Calendar III	Hard
Count of Range Sum	Hard

STICKY NOTES

• Static array => prefix sums; updates too => Fenwick/segment tree
• Segment tree array size: 4n is always safe
• Fenwick: update i += i & -i, query i -= i & -i (1-indexed)
• Lazy propagation = defer updates until a query needs them

DAY 20 · SAT 21 NOV

Advanced Graphs + Union of Patterns

Concepts

- Strongly connected components: Kosaraju & Tarjan
- Bridges and articulation points (low-link values)
- Eulerian path (Hierholzer), bipartite check
- A* search; max-flow intro (Ford-Fulkerson / Edmonds-Karp)
- The 15 coding patterns cheat sheet: map any question to a pattern in 2 minutes
- LCA with binary lifting (O(log n) per query) and Euler tour basics

LIVE JAVA DEMO

Tarjan's algorithm on a sample network: print discovery and low values and highlight the critical connections.

Problems

Is Graph Bipartite?	Medium
Critical Connections in a Network	Hard
Reconstruct Itinerary	Hard
Verifying an Alien Dictionary	Easy
Bus Routes	Hard
Kth Ancestor of a Tree Node	Hard

STICKY NOTES

- Bridge (u,v) when low[v] > disc[u]
- Bipartite = 2-colourable = no odd cycle
- Eulerian path: at most 2 nodes with odd degree
- Pattern first, code second: say the pattern out loud in interviews

DAY 21 · SUN 22 NOV

Week 3 Review + Mock Contest

Concepts

- Virtual LeetCode contest (4 problems, 90 minutes)
- Re-solve every problem you marked 'needed hint' this week
- Revise DP sticky notes; write the state+transition for 10 DP problems without coding
- Java NIO networking: ServerSocketChannel + Selector
- Binary protocol design (length-prefixed messages)

LIVE JAVA DEMO

Connect to NexusKV over TCP from two terminals at once and run SET/GET commands, like redis-cli.

Problems

LFU Cache	Hard
Number of Recent Calls	Easy
Max Points on a Line	Hard

NexusKV build: NexusKV: non-blocking TCP server (Java NIO) with a simple text protocol + client CLI.

STICKY NOTES

- Contest rhythm: read all 4, solve easiest first, never stall more than 20 minutes
- For DP, the state definition is 80% of the solution
- One Selector thread can serve thousands of sockets
- LFU = HashMap + HashMap + minFreq

WEEK 4

LLD, Concurrency & System Design

OOP/SOLID/patterns, Java concurrency, HLD building blocks, distributed systems, mocks

DAY 22 · MON 23 NOV

OOP, SOLID & Design Patterns (Low-Level Design I)

Concepts

- Encapsulation, abstraction, inheritance vs composition, polymorphism
- SOLID with Java examples (and how each one is violated)
- Creational: Singleton (enum/double-checked), Factory, Builder
- Structural: Adapter, Decorator, Proxy, Facade
- Behavioural: Strategy, Observer, Command, State, Chain of Responsibility

LIVE JAVA DEMO

Refactor a 200-line 'god class' payment processor into Strategy + Factory and show how adding UPI becomes a one-class change.

Problems

■ Design Parking System	Easy
■ Design Browser History	Medium
■ Design Underground System	Medium

NexusKV build: NexusKV: refactor command handling into Command pattern + Strategy for eviction policies.

STICKY NOTES

• Favour composition over inheritance
• Open/Closed: add behaviour by adding classes, not editing switch statements
• Enum singleton is serialization- and reflection-safe
• LLD interview flow: requirements -> entities -> relations -> class diagram -> code -> extensibility

DAY 23 · TUE 24 NOV

Java Concurrency + LLD Case Studies

Concepts

- Threads, Runnable/Callable, ExecutorService, CompletableFuture
- synchronized, volatile, happens-before, ReentrantLock, ReadWriteLock
- ConcurrentHashMap, AtomicInteger, BlockingQueue, CountDownLatch
- Deadlock (4 conditions), producer-consumer, thread-safe singleton
- LLD case studies: Parking Lot, Elevator, BookMyShow, Splitwise, Rate Limiter class

LIVE JAVA DEMO

Race condition live: 1,000 threads incrementing a counter (wrong answer) -> fix with synchronized -> AtomicInteger -> LongAdder, with timings.

Problems

■ Print in Order	Easy
■ Print FooBar Alternately	Medium
■ Building H2O	Medium

NexusKV build: NexusKV: thread-safe store (ReadWriteLock / ConcurrentSkipListMap) + JMH benchmark of ops/sec.

STICKY NOTES

- volatile = visibility, NOT atomicity
- Prefer ExecutorService over new Thread()
- ConcurrentHashMap locks per bin, never the whole map
- Deadlock needs: mutual exclusion, hold & wait, no preemption, circular wait

DAY 24 · WED 25 NOV

System Design Fundamentals (HLD I)

Concepts

- The interview framework: requirements -> estimates -> API -> data model -> HLD -> deep dives -> bottlenecks
- Back-of-envelope estimation: QPS, storage, bandwidth
- Vertical vs horizontal scaling, stateless services, load balancers (L4/L7, algorithms)
- Caching: cache-aside, write-through, write-back, eviction, CDN
- CAP & PACELC, consistency models (strong, eventual, read-your-writes)
- API design: REST resources, status codes, pagination (cursor vs offset), versioning, idempotent methods
- Security basics: authentication vs authorisation, sessions vs JWT, OAuth 2.0 flow, HTTPS/TLS
- Distributed unique IDs: UUID vs DB sequence vs Snowflake (timestamp + machine + sequence)

LIVE JAVA DEMO

Run 3 Spring Boot instances behind an Nginx load balancer in Docker Compose; kill one and watch traffic shift.

STICKY NOTES

- 1 day = 86,400 s; 1M requests/day = about 12 QPS
- Cache-aside is the default; always mention invalidation and TTL
- Stateless app servers scale horizontally; state lives in DB/cache
- CAP: under a partition you choose consistency or availability
- Cursor pagination stays correct when rows are inserted; offset pagination does not

DAY 25 · THU 26 NOV

Databases, Storage & Data Distribution (HLD II)

Concepts

- SQL vs NoSQL (key-value, document, wide-column, graph) and when to pick each
- Indexes: B+Tree vs LSM-tree; ACID, isolation levels
- Replication: leader-follower, multi-leader, leaderless (quorum R + W > N)
- Partitioning/sharding: range vs hash; consistent hashing with virtual nodes
- Hot keys, rebalancing, secondary indexes
- SQL interview essentials: joins, GROUP BY/HAVING, window functions, normalisation vs denormalisation
- Blob/object storage (S3-style) vs block storage; storing media metadata separately

LIVE JAVA DEMO

Consistent hashing ring in Java: add/remove nodes and print how many keys move (about 1/N vs almost all with mod N).

NexusKV build: NexusKV: shard keys across 3 nodes with consistent hashing (virtual nodes).

STICKY NOTES

- Consistent hashing moves only about K/N keys when a node joins or leaves
- Quorum: R + W > N gives you overlapping reads/writes
- Read-heavy => replicas + cache; write-heavy => sharding + LSM
- Always state your sharding key and why

DAY 26 · FRI 27 NOV

Distributed Systems & Microservices (HLD III)

Concepts

- Message queues & streams: Kafka partitions, consumer groups, delivery semantics
- Consensus: Raft leader election and log replication
- Rate limiting algorithms: token bucket, leaky bucket, sliding window
- Microservices: API gateway, service discovery, circuit breaker, saga, idempotency keys
- Observability: logs, metrics, tracing; REST vs gRPC vs WebSockets
- Distributed transactions: 2PC vs saga (choreography/orchestration), outbox pattern
- CQRS & event sourcing; search with an inverted index (Elasticsearch)
- Time and ordering: clock skew, vector clocks, at-least-once delivery

LIVE JAVA DEMO

Token-bucket rate limiter in Java hit by a load generator; watch 429s appear exactly at the configured rate.

NexusKV build: NexusKV: leader-follower replication + heartbeat-based failover (Raft-lite leader election).

STICKY NOTES

- Exactly-once is really at-least-once + idempotent consumers
- Raft: a leader needs votes from a majority; terms prevent split brain
- Token bucket allows bursts; leaky bucket smooths them
- Circuit breaker states: closed -> open -> half-open
- Never update a DB and publish an event separately: use the outbox pattern

DAY 27 · SAT 28 NOV

HLD Case Studies

Concepts

- URL shortener (base62, key generation service)
- Rate limiter as a distributed service (Redis + Lua)
- Chat app / WhatsApp (WebSockets, fan-out, message ordering)
- News feed (push vs pull vs hybrid), YouTube/Netflix (CDN, transcoding)
- Uber/ride matching (geohash, quadtree), Google Docs (OT/CRDT overview)
- More case studies: notification system, distributed cache, typeahead/autocomplete, web crawler, Dropbox/Google Drive, ticket booking (BookMyShow)

LIVE JAVA DEMO

Whiteboard-style walkthrough of the URL shortener end to end, then build its core (base62 + store) in 40 lines of Java.

Problems

- Encode and Decode TinyURL — **Medium**
- Search Suggestions System — **Medium**

STICKY NOTES

- Every design: clarify scale first (DAU, read/write ratio, latency target)
- Celebrity problem in feeds => hybrid push/pull
- Geospatial search => geohash or quadtree
- Name trade-offs out loud; there is no single right design
- Ticket booking: lock seats with a TTL hold to avoid double booking

DAY 28 · SUN 29 NOV

NexusKV: Production Polish

Concepts

- Dockerfile + Docker Compose for a 3-node cluster
- GitHub Actions CI: build, test, JaCoCo coverage badge
- JMH benchmarks: throughput and p99 latency tables
- Spring Boot admin dashboard: node health, key counts, ring view
- Architecture diagram + design doc (ADR style)

LIVE JAVA DEMO

docker compose up -> 3 nodes start -> run the benchmark -> kill the leader -> watch failover on the dashboard.

NexusKV build: NexusKV: Docker, CI, benchmarks, dashboard, README with architecture diagram and numbers.

STICKY NOTES

• A project without numbers is a claim; with benchmarks it is evidence
• README order: what, why, architecture, quick start, benchmarks, design decisions
• Keep commits small and meaningful: reviewers read history
• Record a 2-minute demo GIF for the README

DAY 29 · MON 30 NOV

Resume, Behavioural & Company-Specific Prep

Concepts

- Resume bullets with impact: action + what + measurable result
- Amazon Leadership Principles: 2 STAR stories each for the top 8
- Google: Googleyness, structured problem solving, thinking aloud
- Microsoft: growth mindset, collaboration, design-heavy rounds
- Turning Hitachi Energy work into strong stories (sole-developer features, awards)
- Application strategy: referrals, LinkedIn headline + project post, OA preparation (HackerRank/Codility style), offer negotiation basics

LIVE JAVA DEMO

Rewrite 3 weak resume bullets live into quantified, interview-ready bullets.

Problems

- ■ LRU Cache (re-solve in 15 min) — Medium
- ■ Merge k Sorted Lists (re-solve) — Hard
- ■ Course Schedule II (re-solve) — Medium

NexusKV build: NexusKV: final review; pin repo on GitHub; add it to the resume and LinkedIn.

STICKY NOTES

• STAR: keep Situation+Task under 20% of the answer; Result must have a number
• Prepare a 'conflict', a 'failure', and a 'tight deadline' story
• Every resume line is fair game: be ready to go 3 levels deep
• Ask the interviewer a real question at the end

DAY 30 · TUE 01 DEC

Full Mock Interview Day + Final Revision

Concepts

- Mock 1: DSA (2 problems, 45 min, think aloud)
- Mock 2: LLD (design a parking lot or elevator in 45 min)
- Mock 3: HLD (design a URL shortener or chat app in 45 min)
- Mock 4: behavioural (6 questions)
- Revise all 30 days of sticky notes; build the 60-day post-plan

LIVE JAVA DEMO

Recorded mock interview: review the recording for silence gaps, missed edge cases and complexity statements.

Problems

Word Ladder (timed)	Hard
Longest Increasing Path in a Matrix	Hard
Design a Food Rating System	Medium

STICKY NOTES

• Interview loop: clarify -> examples -> brute force -> optimise -> code -> test -> complexity
• Test with an edge case before saying 'done'
• Keep solving 2 problems a day after Day 30: consistency beats cramming
• You are ready when you can explain WHY a pattern applies, not just recall it

FLAGSHIP PROJECT

NexusKV: A distributed, fault-tolerant key-value store built from scratch in Java

It is built out of the exact data structures interviewers ask about (skip list, hash map, LRU, heap, Bloom filter, consistent hashing ring) and the exact system-design topics they ask about (LSM storage, WAL, sharding, replication, leader election). Every interview question about your resume becomes a chance to show depth.

Architecture

```
Client CLI → NIO TCP server → Router (consistent-hash ring) → Leader node → [WAL → Skip-list memtable → SSTables + Bloom filters → Compaction] → replicated to Follower nodes (heartbeats + leader election). Spring Boot admin dashboard reads node health and metrics.
```

Features

Storage engine	LSM-tree: skip-list memtable, WAL, immutable SSTables with index + Bloom filter, background compaction
Caching	O(1) LRU read cache (HashMap + doubly linked list), pluggable LFU strategy
Networking	Non-blocking Java NIO server, length-prefixed protocol, CLI client
Distribution	Consistent hashing with virtual nodes across 3+ nodes
Fault tolerance	Leader-follower replication, heartbeats, Raft-lite leader election, crash recovery from WAL
Engineering	Docker Compose cluster, CI pipeline, test coverage, JMH throughput & p99 latency numbers, admin dashboard

Tech stack

Java 21 · Maven · Java NIO (TCP server) · Spring Boot (admin dashboard + REST) · JUnit 5 + Mockito · JMH benchmarks · Docker + Docker Compose · GitHub Actions CI

Build timeline

- **Day 1:** Create the GitHub repo + Maven multi-module skeleton for NexusKV; set up JUnit 5.
- **Day 6:** implement the LRU read cache (HashMap + DLL) with unit tests.
- **Day 7:** in-memory store with GET/PUT/DELETE and a skip-list memtable (ordered keys).
- **Day 10:** K-way merge iterator (heap) to merge sorted runs.
- **Day 11:** Bloom filter per data file to skip disk reads for missing keys.
- **Day 14:** WAL + flush memtable to sorted SSTable files with an index block and Bloom filter.
- **Day 18:** background compaction (merge SSTables with the heap iterator, drop tombstones).
- **Day 21:** non-blocking TCP server (Java NIO) with a simple text protocol + client CLI.
- **Day 22:** refactor command handling into Command pattern + Strategy for eviction policies.
- **Day 23:** thread-safe store (ReadWriteLock / ConcurrentSkipListMap) + JMH benchmark of ops/sec.
- **Day 25:** shard keys across 3 nodes with consistent hashing (virtual nodes).
- **Day 26:** leader-follower replication + heartbeat-based failover (Raft-lite leader election).
- **Day 28:** Docker, CI, benchmarks, dashboard, README with architecture diagram and numbers.
- **Day 29:** final review; pin repo on GitHub; add it to the resume and LinkedIn.

Resume bullets (fill in your benchmark numbers)

- Built NexusKV, a distributed key-value store in Java 21 with an LSM-tree storage engine (skip-list memtable, WAL, SSTables, Bloom filters), sustaining ~XXk ops/sec at p99 < X ms in JMH benchmarks.
- Designed sharding with consistent hashing (virtual nodes) and leader-follower replication with automatic failover, recovering from node crashes with zero acknowledged-write loss.

- Implemented a non-blocking Java NIO server, O(1) LRU cache and background compaction; shipped with Docker Compose, GitHub Actions CI and 85%+ test coverage.

Lighter alternative if time runs short: a Spring Boot microservices URL shortener + distributed rate limiter (Redis, Docker, API gateway). Less impressive, but finishable in 4 weekends.

Extra topics included (beyond pure DSA)

Java Collections internals	HashMap, ArrayList, ArrayDeque, PriorityQueue, TreeMap, ConcurrentHashMap: how they work inside (asked often at Amazon/Microsoft).
Java concurrency	Day 23. Needed for LLD rounds and for the project.
15 coding patterns	Two pointers, sliding window, fast/slow, merge intervals, cyclic sort, in-place reversal, BFS, DFS, two heaps, subsets, modified binary search, top-K, K-way merge, topological sort, DP.
Low-Level Design	Days 22-23. A separate round at Amazon, Microsoft, Flipkart, Uber.
Behavioural rounds	Day 29. Amazon weights Leadership Principles heavily in every round.
Docker & CI/CD basics	Learned through the project on Day 28 (fills a gap on your profile).
Mock interviews	Days 21 and 30. Use Pramp / interviewing.io peers or record yourself.

Interview day framework

- **DSA:** clarify → examples → brute force → optimise → code → dry run → complexity
- **LLD:** requirements → entities → relationships → class diagram → code core classes → patterns & extensibility
- **HLD:** requirements → estimates → API → data model → high-level diagram → deep dives → bottlenecks & trade-offs
- **Behavioural:** STAR, with a number in every Result

Resources

- **LeetCode**: Every problem in this plan links here
- **NeetCode 150 roadmap**: Pattern-wise problem list
- **VisuAlgo**: Animations of every data structure
- **System Design Primer**: Free HLD reference
- **Designing Data-Intensive Applications**: The book behind NexusKV's design (Ch. 3, 5, 6, 9)
- **Refactoring Guru**: Design patterns with Java examples

After Day 30

- Keep 2 problems/day (one new, one re-solve) until your interviews.
- One HLD case study per week from the list on Day 27.
- Weekly LeetCode contest every Sunday morning.

APPENDIX A

20 coding patterns: spot the pattern in 2 minutes

Pattern	Signal in the question	Classic problem
Two pointers	Sorted array, pair/triplet sums, in-place removal	3Sum
Sliding window	Longest/shortest contiguous subarray or substring with a condition	Minimum Window Substring
Fast & slow pointers	Cycles, middle of list, 'happy number'	Linked List Cycle II
Prefix sum + hash map	Count subarrays with a given sum/property	Subarray Sum Equals K
Cyclic sort	Numbers in range 1..n, find missing/duplicate	First Missing Positive
Merge intervals	Overlapping ranges, meetings, calendars	Merge Intervals
In-place list reversal	Reverse a list or a sub-part of it	Reverse Nodes in k-Group
Monotonic stack/deque	Next greater/smaller, spans, window max	Largest Rectangle in Histogram
Tree DFS / BFS	Paths, depths, level views, LCA	Binary Tree Maximum Path Sum
Graph BFS / DFS	Grids, islands, shortest unweighted path	Rotting Oranges
Topological sort	Dependencies, ordering, course prerequisites	Course Schedule II
Union-Find	Dynamic connectivity, grouping, redundant edges	Number of Provinces
Two heaps	Running median, balancing two halves	Find Median from Data Stream
Top-K / K-way merge	K largest/closest/frequent, merging sorted lists	Merge k Sorted Lists
Modified binary search	Sorted/rotated arrays, min/max feasible answer	Koko Eating Bananas
Backtracking	All subsets, permutations, combinations, boards	N-Queens
Dynamic programming	Count ways / min-max cost with overlapping choices	Edit Distance
Greedy	Locally best choice provably optimal (sort first)	Non-overlapping Intervals
Trie	Prefix search, autocomplete, many-word matching	Word Search II
Bit manipulation	XOR tricks, subsets as masks, n <= 20	Single Number

APPENDIX B

What the interview loops look like (typical, varies by team and level)

Google	Online assessment or phone screen, then about 4 to 5 interviews: mostly DSA (medium-hard, often graphs/DP), one Googleyness & leadership round, system design for L4+. Clean code and thinking aloud matter; a hiring committee decides.
Amazon	Online assessment (2 coding problems + work-style survey), then a loop of about 4 rounds: DSA, LLD/HLD for SDE-2, and Leadership Principles questions in every round. A Bar Raiser interviewer has veto power.
Microsoft	Online assessment or screen, then about 4 rounds: DSA, LLD/HLD (often a full design round for 3+ years), and an 'as appropriate' final round with a senior manager. Collaboration and growth mindset are probed.

APPENDIX C

Java DSA syntax cheat sheet

Stack / Queue

```
Deque<Integer> st = new ArrayDeque<>(); st.push(x); st.pop(); st.peek();
Queue<Integer> q = new ArrayDeque<>(); q.offer(x); q.poll();
```

Heaps

```
PriorityQueue<Integer> min = new PriorityQueue<>();
PriorityQueue<Integer> max = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
```

Maps

```
map.merge(k, 1, Integer::sum); map.getOrDefault(k, 0);
map.computeIfAbsent(k, x -> new ArrayList<>()).add(v);
```

TreeMap

```
tm.floorKey(x); tm.ceilingKey(x); tm.firstKey(); tm.pollFirstEntry(); tm.headMap(x);
```

Sorting

```
Arrays.sort(a); Arrays.sort(iv, (x, y) -> Integer.compare(x[0], y[0]));
list.sort(Comparator.comparing(Person::age).thenComparing(Person::name));
```

Strings

```
char[] c = s.toCharArray(); new String(c); String.valueOf(c);
StringBuilder sb = new StringBuilder(); sb.append(x).reverse().toString();
```

Arrays

```
int[] a = new int[n]; Arrays.fill(a, -1); int[][] g = new int[r][c];
int[] copy = Arrays.copyOfRange(a, l, r); Arrays.toString(a);
```

Overflow & math

```
long x = (long) a * b; Integer.MAX_VALUE; Math.floorMod(a, m);
int mid = lo + (hi - lo) / 2; Integer.bitCount(x);
```

APPENDIX D

Revision system

- Spaced repetition: re-solve each problem you needed help on after 2 days, 7 days and 21 days.
- Every Sunday (Days 7, 14, 21, 28): 30 minutes reading all sticky notes of the week.
- Keep a mistake log (problem, what tripped you, the fix) in the GitHub repo.
- Days 29-30: read all 30 sticky notes and the patterns table once more.